

We can use that texture on some spheres:

```
void perlin_spheres() {
    hittable_list world;

    auto pertext = make_shared<noise_texture>();
    world.add(make_shared<sphere>(point3(0,-1000,0), 1000, make_shared<lambertian>
(pertext)));
    world.add(make_shared<sphere>(point3(0,2,0), 2, make_shared<lambertian>
(pertext)));

    camera cam;

    cam.aspect_ratio      = 16.0 / 9.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth         = 50;

    cam.vfov              = 20;
    cam.lookfrom = point3(13,2,3);
    cam.lookat   = point3(0,0,0);
    cam.vup      = vec3(0,1,0);

    cam.defocus_angle = 0;

    cam.render(world);
}

int main() {
    switch (4) {
        case 1: bouncing_spheres(); break;
        case 2: checkered_spheres(); break;
        case 3: earth(); break;
        case 4: perlin_spheres(); break;
    }
}
```

Listing 36: [main.cc] *Scene with two Perlin-textured spheres*

And the hashing does scramble as hoped:

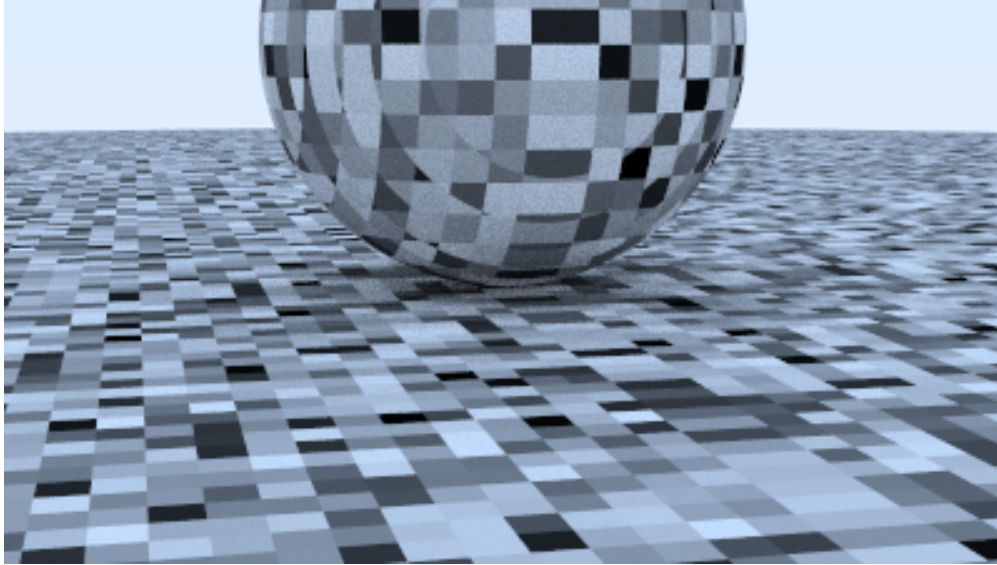


Image 9: *Hashed random texture*

5.2. Smoothing out the Result

To make it smooth, we can linearly interpolate:

```

class perlin {
public:
    ...

    double noise(const point3& p) const {
        auto u = p.x() - std::floor(p.x());
        auto v = p.y() - std::floor(p.y());
        auto w = p.z() - std::floor(p.z());

        auto i = int(std::floor(p.x()));
        auto j = int(std::floor(p.y()));
        auto k = int(std::floor(p.z()));
        double c[2][2][2];

        for (int di=0; di < 2; di++)
            for (int dj=0; dj < 2; dj++)
                for (int dk=0; dk < 2; dk++)
                    c[di][dj][dk] = randfloat[
                        perm_x[(i+di) & 255] ^
                        perm_y[(j+dj) & 255] ^
                        perm_z[(k+dk) & 255]
                    ];

        return trilinear_interp(c, u, v, w);
    }

    ...

private:
    ...

    static void permute(int* p, int n) {
        ...
    }

    static double trilinear_interp(double c[2][2][2], double u, double v, double w) {
        auto accum = 0.0;
        for (int i=0; i < 2; i++)
            for (int j=0; j < 2; j++)
                for (int k=0; k < 2; k++)
                    accum += (i*u + (1-i)*(1-u))
                        * (j*v + (1-j)*(1-v))
                        * (k*w + (1-k)*(1-w))
                        * c[i][j][k];

        return accum;
    }
};

```

Listing 37: [perlin.h] *Perlin with trilinear interpolation*

And we get:

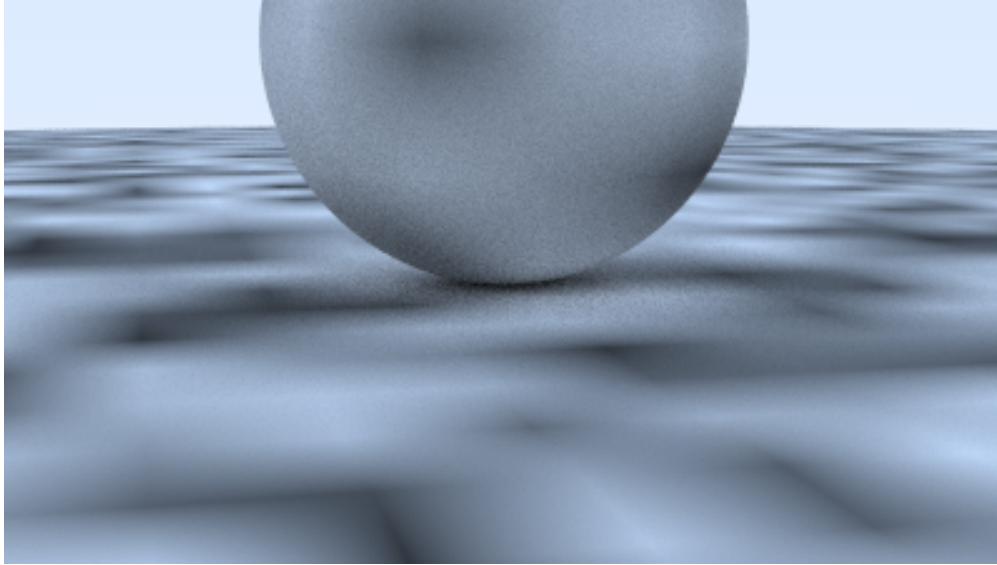


Image 10: *Perlin texture with trilinear interpolation*

5.3. Improvement with Hermitian Smoothing

Smoothing yields an improved result, but there are obvious grid features in there. Some of it is Mach bands, a known perceptual artifact of linear interpolation of color. A standard trick is to use a Hermite cubic to round off the interpolation:

```
class perlin (  
    public:  
        ...  
        double noise(const point3& p) const {  
            auto u = p.x() - std::floor(p.x());  
            auto v = p.y() - std::floor(p.y());  
            auto w = p.z() - std::floor(p.z());  
            u = u*u*(3-2*u);  
            v = v*v*(3-2*v);  
            w = w*w*(3-2*w);  
  
            auto i = int(std::floor(p.x()));  
            auto j = int(std::floor(p.y()));  
            auto k = int(std::floor(p.z()));  
            ...  
        }  
};
```

Listing 38: `[perlin.h]` *Perlin with Hermitian smoothing*

This gives a smoother looking image:

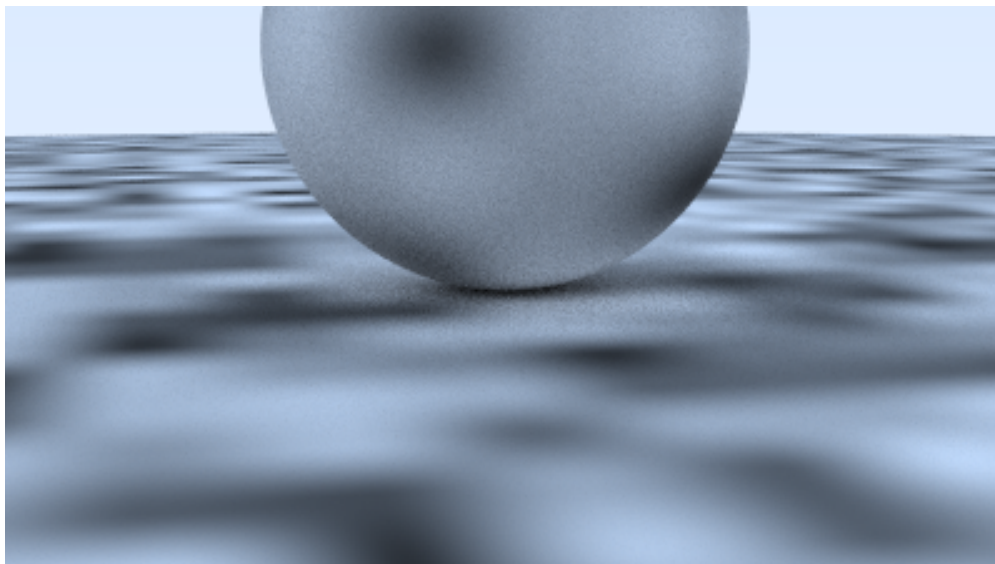


Image 11: *Perlin texture, trilinearly interpolated, smoothed*

5.4. Tweaking The Frequency

It is also a bit low frequency. We can scale the input point to make it vary more quickly:

```
class noise_texture : public texture {
public:
    noise_texture(double scale) : scale(scale) {}

    color value(double u, double v, const point3& p) const override {
        return color(1,1,1) * noise.noise(scale * p);
    }

private:
    perlin noise;
    double scale;
};
```

Listing 39: [texture.h] *Perlin smoothed, higher frequency*

We then add that scale to the `perlin_spheres()` scene description:

```
void perlin_spheres() {  
    ...  
    auto pertext = make_shared<noise_texture>(4);  
    world.add(make_shared<sphere>(point3(0, -1000, 0), 1000, make_shared<lambertian>  
    (pertext)));  
    world.add(make_shared<sphere>(point3(0, 2, 0), 2, make_shared<lambertian>  
    (pertext)));  
  
    camera cam;  
    ...  
}
```

Listing 40: `[main.cc]` *Perlin-textured spheres with a scale to the noise*

This yields the following result:

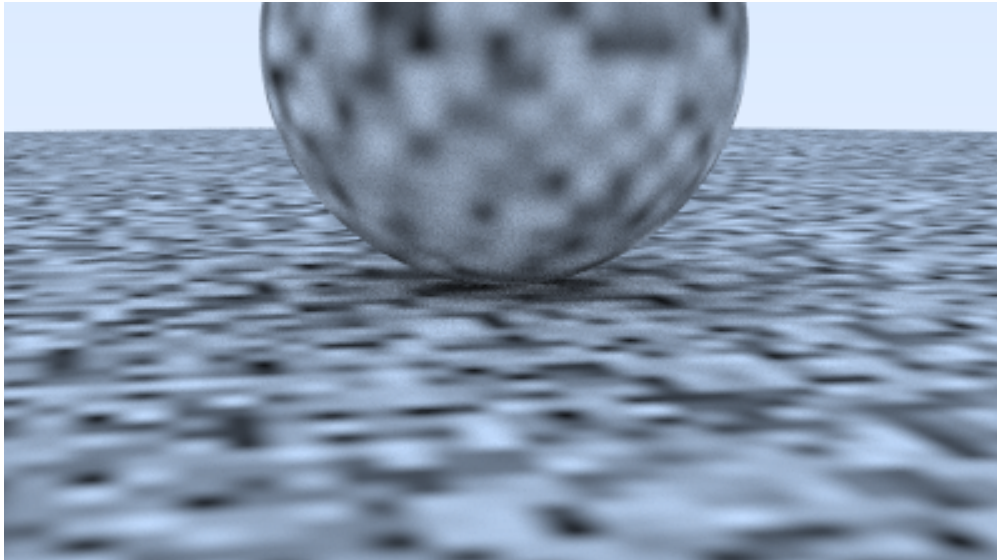


Image 12: *Perlin texture, higher frequency*

5.5. Using Random Vectors on the Lattice Points

This is still a bit blocky looking, probably because the min and max of the pattern always lands exactly on the integer $x/y/z$. Ken Perlin's very clever trick was to instead put random unit vectors (instead of just floats) on the lattice points, and use a dot product to move the min and max off the lattice. So, first we need to change the random floats to random vectors. These vectors are any reasonable set of irregular directions, and I won't bother to make them exactly uniform:

```
class perlin {  
public:  
    perlin() {  
        for (int i = 0; i < point_count; i++) {  
            randvec[i] = unit_vector(vec3::random(-1,1));  
        }  
  
        perlin_generate_perm(perm_x);  
        perlin_generate_perm(perm_y);  
        perlin_generate_perm(perm_z);  
    }  
  
    ...  
  
private:  
    static const int point_count = 256;  
    vec3 randvec[point_count];  
    int perm_x[point_count];  
    int perm_y[point_count];  
    int perm_z[point_count];  
    ...  
};
```

Listing 41: [perlin.h] *Perlin with random unit translations*

The Perlin class `noise()` method is now:

```
class perlin {
public:
    ...
    double noise(const point3& p) const {
        auto u = p.x() - std::floor(p.x());
        auto v = p.y() - std::floor(p.y());
        auto w = p.z() - std::floor(p.z());

        u = u*u*(3-2*u);
        v = v*v*(3-2*v);
        w = w*w*(3-2*w);

        auto i = int(std::floor(p.x()));
        auto j = int(std::floor(p.y()));
        auto k = int(std::floor(p.z()));
        vec3 c[2][2][2];

        for (int di=0; di < 2; di++)
            for (int dj=0; dj < 2; dj++)
                for (int dk=0; dk < 2; dk++)
                    c[di][dj][dk] = randvec[
                        perm_x[(i+di) & 255] ^
                        perm_y[(j+dj) & 255] ^
                        perm_z[(k+dk) & 255]
                    ];

        return perlin_interp(c, u, v, w);
    }
    ...
};
```

Listing 42: `[perlin.h]` *Perlin class with new `noise()` method*

And the interpolation becomes a bit more complicated:

```
class perlin {
...
private:
...
static double trilinear_interp(double c[2][2][2], double u, double v, double w) {
...
}

static double perlin_interp(const vec3 c[2][2][2], double u, double v, double w)
{
    auto uu = u*u*(3-2*u);
    auto vv = v*v*(3-2*v);
    auto ww = w*w*(3-2*w);
    auto accum = 0.0;

    for (int i=0; i < 2; i++)
        for (int j=0; j < 2; j++)
            for (int k=0; k < 2; k++) {
                vec3 weight_v(u-i, v-j, w-k);
                accum += (i*uu + (1-i)*(1-uu))
                    * (j*vv + (1-j)*(1-vv))
                    * (k*ww + (1-k)*(1-ww))
                    * dot(c[i][j][k], weight_v);
            }

    return accum;
}
};
```

Listing 43: [perlin.h] *Perlin interpolation function so far*

The output of the Perlin interpolation function can return negative values. These negative values will be passed to our `linear_to_gamma()` color function, which expects only positive inputs. To mitigate this, we'll map the $[-1, +1]$ range of values to $[0, 1]$.

```
class noise_texture : public texture {
public:
    noise_texture(double scale) : scale(scale) {}

    color value(double u, double v, const point3& p) const override {
        return color(1,1,1) * 0.5 * (1.0 + noise.noise(scale * p));
    }

private:
    perlin noise;
    double scale;
};
```

Listing 44: [texture.h] *Perlin smoothed, higher frequency*

This finally gives something more reasonable looking:

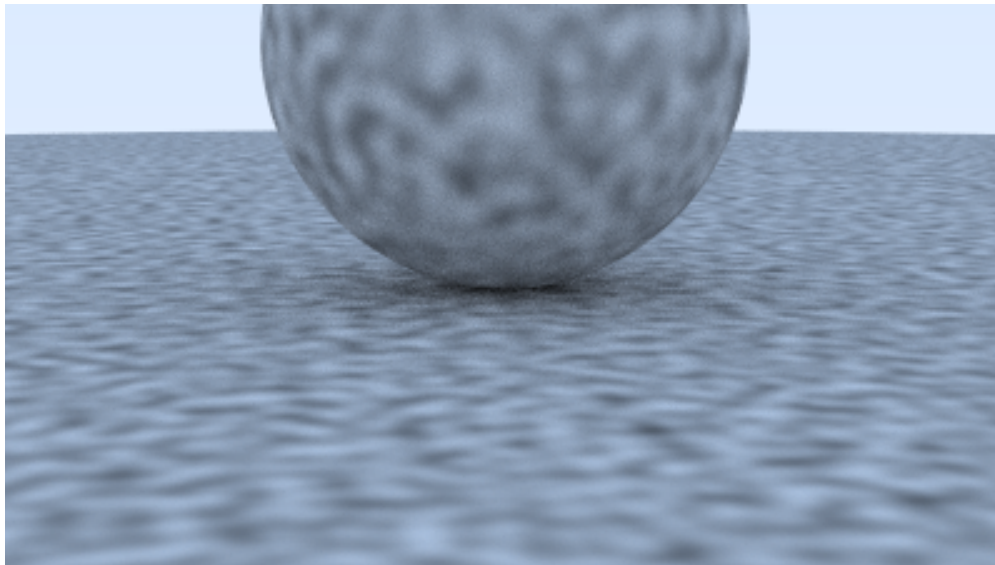


Image 13: *Perlin texture, shifted off integer values*

5.6. Introducing Turbulence

Very often, a composite noise that has multiple summed frequencies is used. This is usually called turbulence, and is a sum of repeated calls to noise:

```
class perlin {  
    ...  
public:  
    ...  
  
    double noise(const point3& p) const {  
        ...  
    }  
  
    double turb(const point3& p, int depth) const {  
        auto accum = 0.0;  
        auto temp_p = p;  
        auto weight = 1.0;  
  
        for (int i = 0; i < depth; i++) {  
            accum += weight * noise(temp_p);  
            weight *= 0.5;  
            temp_p *= 2;  
        }  
  
        return std::fabs(accum);  
    }  
    ...  
}
```

Listing 45: [perlin.h] *Turbulence function*

```

class noise_texture : public texture {
public:
    noise_texture(double scale) : scale(scale) {}

    color value(double u, double v, const point3& p) const override {
        return color(1,1,1) * noise.turb(p, 7);
    }

private:
    perlin noise;
    double scale;
};

```

Listing 46: [texture.h] *Noise texture with turbulence*

Used directly, turbulence gives a sort of camouflage netting appearance:

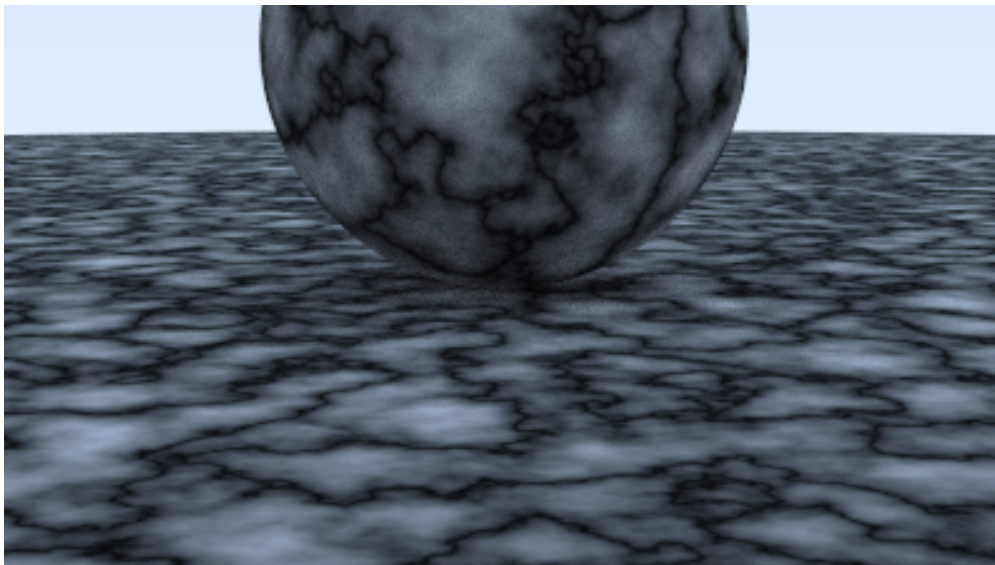


Image 14: *Perlin texture with turbulence*

5.7. Adjusting the Phase

However, usually turbulence is used indirectly. For example, the “hello world” of procedural solid textures is a simple marble-like texture. The basic idea is to make color proportional to something like a sine function, and use turbulence to adjust the phase (so it shifts x in $\sin(x)$) which makes the stripes undulate. Commenting out straight noise and turbulence, and giving a marble-like effect is:

```
class noise_texture : public texture {  
public:  
    noise_texture(double scale) : scale(scale) {}  
  
    color value(double u, double v, const point3& p) const override {  
        return color(.5, .5, .5) * (1 + std::sin(scale * p.z() + 10 * noise.turb(p,  
7)));  
    }  
  
private:  
    perlin noise;  
    double scale;  
};
```

Listing 47: [texture.h] *Noise texture with marbled texture*

Which yields:

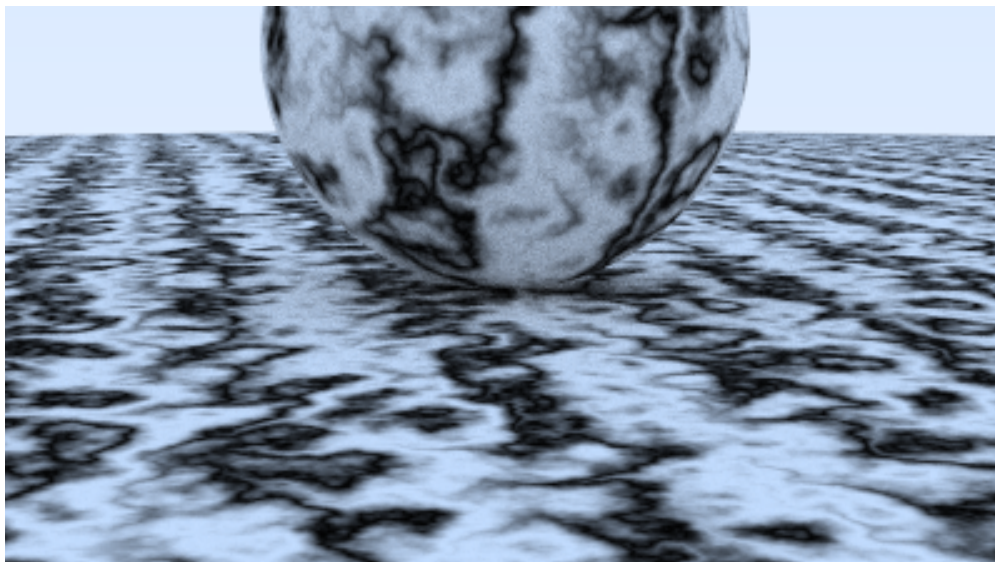


Image 15: *Perlin noise, marbled texture*

6. Quadrilaterals

We've managed to get more than half way through this three-book series using spheres as our only geometric primitive. Time to add our second primitive: the quadrilateral.

6.1. Defining the Quadrilateral

Though we'll name our new primitive a **quad**, it will technically be a parallelogram (opposite sides are parallel) instead of a general quadrilateral. For our purposes, we'll use three geometric entities to define a quad:

1. $\mathbf{Q}$, the starting corner.
2. $\mathbf{u}$, a vector representing the first side. $\mathbf{Q} + \mathbf{u}$ gives one of the corners adjacent to $\mathbf{Q}$.
3. $\mathbf{v}$, a vector representing the second side. $\mathbf{Q} + \mathbf{v}$ gives the other corner adjacent to $\mathbf{Q}$.

The corner of the quad opposite $\mathbf{Q}$ is given by $\mathbf{Q} + \mathbf{u} + \mathbf{v}$. These values are three-dimensional, even though a quad itself is a two-dimensional object. For example, a quad with corner at the origin and extending two units in the Z direction and one unit in the Y direction would have values $\mathbf{Q} = (0, 0, 0)$, $\mathbf{u} = (0, 0, 2)$, and $\mathbf{v} = (0, 1, 0)$.

The following figure illustrates the quadrilateral components.

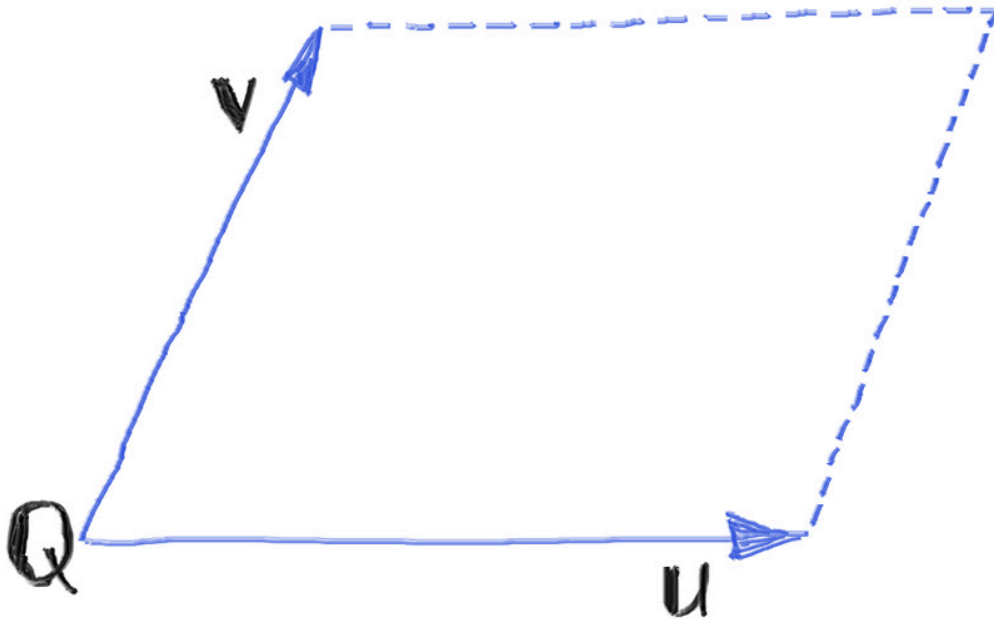


Figure 5: Quadrilateral Components

Quads are flat, so their axis-aligned bounding box will have zero thickness in one dimension if the quad lies in the XY, YZ, or ZX plane. This can lead to numerical problems with ray intersection, but we can address this by padding any zero-sized dimensions of the bounding box. Padding is fine because we aren't changing the intersection of the quad; we're only expanding its bounding box to remove the possibility of numerical problems, and the bounds are just a rough approximation to the actual shape anyway. To remedy this situation, we insert a small padding to ensure that newly constructed AABBs always have a non-zero volume:

```
...
class aabb {
public:
    ...
    aabb(const interval& x, const interval& y, const interval& z)
        : x(x), y(y), z(z)
    {
        pad_to_minimums();
    }

    aabb(const point3& a, const point3& b) {
        // Treat the two points a and b as extrema for the bounding box, so we don't
        // require a
        // particular minimum/maximum coordinate order.

        x = interval(std::fmin(a[0],b[0]), std::fmax(a[0],b[0]));
        y = interval(std::fmin(a[1],b[1]), std::fmax(a[1],b[1]));
        z = interval(std::fmin(a[2],b[2]), std::fmax(a[2],b[2]));

        pad_to_minimums();
    }
    ...
    static const aabb empty, universe;

private:
    void pad_to_minimums() {
        // Adjust the AABBB so that no side is narrower than some delta, padding if
        // necessary.

        double delta = 0.0001;
        if (x.size() < delta) x = x.expand(delta);
        if (y.size() < delta) y = y.expand(delta);
        if (z.size() < delta) z = z.expand(delta);
    }
};
```

Listing 48: [aabb.h] *New aabb::pad_to_minimums() method*

Now we're ready for the first sketch of the new `quad` class:

```
#ifndef QUAD_H
#define QUAD_H

#include "hitable.h"

class quad : public hittable {
public:
    quad(const point3& Q, const vec3& u, const vec3& v, shared_ptr<material> mat)
        : Q(Q), u(u), v(v), mat(mat)
    {
        set_bounding_box();
    }

    virtual void set_bounding_box() {
        // Compute the bounding box of all four vertices.
        auto bbox_diagonal1 = aabb(Q, Q + u + v);
        auto bbox_diagonal2 = aabb(Q + u, Q + v);
        bbox = aabb(bbox_diagonal1, bbox_diagonal2);
    }

    aabb bounding_box() const override { return bbox; }

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        return false; // To be implemented
    }

private:
    point3 Q;
    vec3 u, v;
    shared_ptr<material> mat;
    aabb bbox;
};

#endif
```

Listing 49: `[quad.h]` 2D quadrilateral (parallelogram) class

6.2. Ray-Plane Intersection

As you can see in the prior listing, `quad::hit()` remains to be implemented. Just as for spheres, we need to determine whether a given ray intersects the primitive, and if so, the various properties of that intersection (hit point, normal, texture coordinates and so forth).

Ray-quad intersection will be determined in three steps:

1. finding the plane that contains that quad,
2. solving for the intersection of a ray and the quad-containing plane,
3. determining if the hit point lies inside the quad.

We'll first tackle the middle step, solving for general ray-plane intersection.

Spheres are generally the first ray tracing primitive taught because their implicit formula makes it so easy to solve for ray intersection. Like spheres, planes also have an implicit formula, and we can use their implicit formula to produce an algorithm that solves for ray-plane intersection. Indeed, ray-plane intersection is even *easier* to solve than ray-sphere intersection.

You may already know this implicit formula for a plane:

$$Ax + By + Cz + D = 0$$

where A, B, C, D are just constants, and x, y, z are the values of any point (x, y, z) that lies on the plane. A plane is thus the set of all points (x, y, z) that satisfy the formula above. It makes things slightly easier to use the alternate formulation:

$$Ax + By + Cz = D$$

(We didn't flip the sign of D because it's just some constant that we'll figure out later.)

Here's an intuitive way to think of this formula: given the plane perpendicular to the normal vector $\mathbf{n} = (A, B, C)$, and the position vector $\mathbf{v} = (x, y, z)$ (that is, the vector from the origin to any point on the plane), then we can use the dot product to solve for D :

$$\mathbf{n} \cdot \mathbf{v} = D$$

for any position on the plane. This is an equivalent formulation of the $Ax + By + Cz = D$ formula given above, only now in terms of vectors.

Now to find the intersection with some ray $\mathbf{R}(t) = \mathbf{P} + t\mathbf{d}$. Plugging in the ray equation, we get

$$\mathbf{n} \cdot (\mathbf{P} + t\mathbf{d}) = D$$

Solving for t :

$$\mathbf{n} \cdot \mathbf{P} + \mathbf{n} \cdot t\mathbf{d} = D$$

$$\mathbf{n} \cdot \mathbf{P} + t(\mathbf{n} \cdot \mathbf{d}) = D$$

$$t = \frac{D - \mathbf{n} \cdot \mathbf{P}}{\mathbf{n} \cdot \mathbf{d}}$$

This gives us t , which we can plug into the ray equation to find the point of intersection. Note that the denominator $\mathbf{n} \cdot \mathbf{d}$ will be zero if the ray is parallel to the plane. In this case, we can immediately record a miss between the ray and the plane. As for other primitives, if the ray t parameter is less than the minimum acceptable value, we also record a miss.

All right, we can find the point of intersection between a ray and the plane that contains a given quadrilateral. In fact, we can use this approach to test *any* planar primitive, like triangles and disks (more on that later).

6.3. Finding the Plane That Contains a Given Quadrilateral

We've solved step two above: solving the ray-plane intersection, assuming we have the plane equation. To do this, we need to tackle step one above: finding the equation for the plane that contains the quad. We have quadrilateral parameters $\mathbf{Q}$, $\mathbf{u}$, and $\mathbf{v}$, and want the corresponding equation of the plane containing the quad defined by these three values.

Fortunately, this is very simple. Recall that in the equation $Ax + By + Cz = D$, (A, B, C) represents the normal vector. To get this, we just use the cross product of the two side vectors $\mathbf{u}$ and $\mathbf{v}$:

$$\mathbf{n} = \text{unit_vector}(\mathbf{u} \times \mathbf{v})$$

The plane is defined as all points (x, y, z) that satisfy the equation $Ax + By + Cz = D$. Well, we know that $\mathbf{Q}$ lies on the plane, so that's enough to solve for D :

$$\begin{aligned} D &= n_x Q_x + n_y Q_y + n_z Q_z \\ &= \mathbf{n} \cdot \mathbf{Q} \end{aligned}$$

Add the planar values to the `quad` class:

```
class quad : public hittable {
public:
    quad(const point3& Q, const vec3& u, const vec3& v, shared_ptr<material> mat)
        : Q(Q), u(u), v(v), mat(mat)
    {
        auto n = cross(u, v);
        normal = unit_vector(n);
        D = dot(normal, Q);

        set_bounding_box();
    }
    ...

private:
    point3 Q;
    vec3 u, v;
    shared_ptr<material> mat;
    aabb bbox;
    vec3 normal;
    double D;
};
```

Listing 50: [quad. h] *Caching the planar values*

We will use the two values `normal` and `D` to find the point of intersection between a given ray and the plane containing the quadrilateral.

As an incremental step, let's implement the `hit()` method to handle the infinite plane containing our quadrilateral.

```

class quad : public hittable {
    ...

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        auto denom = dot(normal, r.direction());

        // No hit if the ray is parallel to the plane.
        if (std::fabs(denom) < 1e-8)
            return false;

        // Return false if the hit point parameter t is outside the ray interval.
        auto t = (D - dot(normal, r.origin())) / denom;
        if (!ray_t.contains(t))
            return false;

        auto intersection = r.at(t);

        rec.t = t;
        rec.p = intersection;
        rec.mat = mat;
        rec.set_face_normal(r, normal);

        return true;
    }

    ...
}

```

Listing 51: [quad.h] *hit()* method for the infinite plane

6.4. Orienting Points on The Plane

At this stage, the intersection point is on the plane that contains the quadrilateral, but it could be *anywhere* on the plane: the ray-plane intersection point will lie inside or outside the quadrilateral. We need to test for intersection points that lie inside the quadrilateral (hit), and reject points that lie outside (miss). To determine where a point lies relative to the quad, and to assign texture coordinates to the point of intersection, we need to orient the intersection point on the plane.

To do this, we'll construct a *coordinate frame* for the plane — a way of orienting any point located on the plane. We've already been using a coordinate frame for our 3D space — this is defined by an origin point **O** and three basis vectors **x**, **y**, and **z**.

Since a plane is a 2D construct, we just need a plane origin point **Q** and *two* basis vectors: **u** and **v**. Normally, axes are perpendicular to each other. However, this doesn't need to be the case in order to span the entire space — you just need two axes that are not parallel to each other.

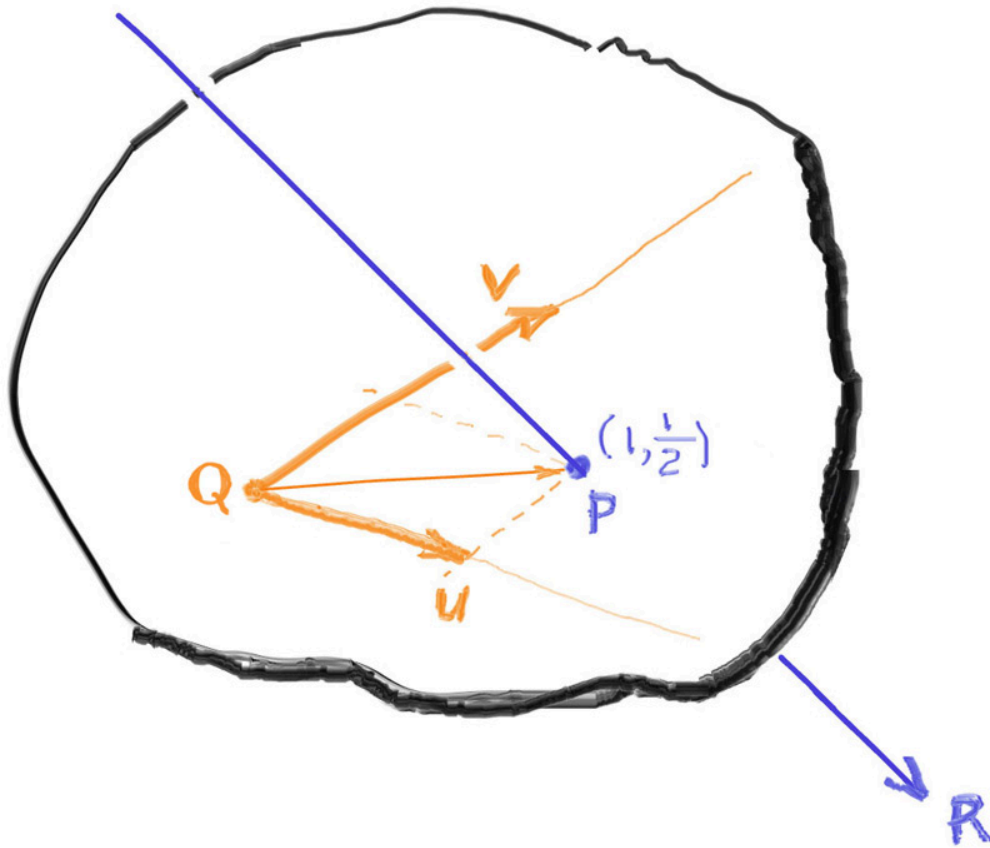


Figure 6: Ray-plane intersection

Consider figure 6 as an example. Ray **R** intersects the plane, yielding intersection point **P** (not to be confused with the ray origin point **P** above). Measuring against plane vectors **u** and **v**, the intersection point **P** in the example above is at $\mathbf{Q} + (1)\mathbf{u} + (\frac{1}{2})\mathbf{v}$. In other words, the **UV** (plane) coordinates of intersection point **P** are $(1, \frac{1}{2})$.

Generally, given some arbitrary point **P**, we seek two scalar values α and β , so that

$$\mathbf{P} = \mathbf{Q} + \alpha\mathbf{u} + \beta\mathbf{v}$$

Pulling a rabbit out of my hat, the planar coordinates α and β are given by the following equations:

$$\alpha = \mathbf{w} \cdot (\mathbf{p} \times \mathbf{v})$$

$$\beta = \mathbf{w} \cdot (\mathbf{u} \times \mathbf{p})$$

where

$$\mathbf{p} = \mathbf{P} - \mathbf{Q}$$

$$\mathbf{w} = \frac{\mathbf{n}}{\mathbf{n} \cdot (\mathbf{u} \times \mathbf{v})} = \frac{\mathbf{n}}{\mathbf{n} \cdot \mathbf{n}}$$

The vector **w** is constant for a given quadrilateral, so we'll cache that value.

```

class quad : public hittable {
public:
    quad(const point3& Q, const vec3& u, const vec3& v, shared_ptr<material> mat)
        : Q(Q), u(u), v(v), mat(mat)
    {
        auto n = cross(u, v);
        normal = unit_vector(n);
        D = dot(normal, Q);
        w = n / dot(n,n);

        set_bounding_box();
    }

    ...

private:
    point3 Q;
    vec3 u, v;
    vec3 w;
    shared_ptr<material> mat;
    aabb bbox;
    vec3 normal;
    double D;
};

```

Listing 52: [quad. h] *Caching the quadrilateral's w value*

6.5. Deriving the Planar Coordinates

(This section covers the derivation of the equations above. Feel free to skip to the next section if you're not interested.)

Refer back to [figure 6](#). If the planar basis vectors $\mathbf{u}$ and $\mathbf{v}$ were guaranteed to be orthogonal to each other (forming a 90° angle between them), then solving for α and β would be a simple matter of using the dot product to project $\mathbf{P}$ onto each of the basis vectors $\mathbf{u}$ and $\mathbf{v}$. However, since we are not restricting $\mathbf{u}$ and $\mathbf{v}$ to be orthogonal, the math's a little bit trickier.

To set things up, consider that

$$\mathbf{P} = \mathbf{Q} + \alpha\mathbf{u} + \beta\mathbf{v}$$

$$\mathbf{p} = \mathbf{P} - \mathbf{Q} = \alpha\mathbf{u} + \beta\mathbf{v}$$

Here, $\mathbf{P}$ is the *point* of intersection, and $\mathbf{p}$ is the *vector* from $\mathbf{Q}$ to $\mathbf{P}$.

Cross the equation for $\mathbf{p}$ with $\mathbf{u}$ and $\mathbf{v}$, respectively:

$$\begin{aligned}
 \mathbf{u} \times \mathbf{p} &= \mathbf{u} \times (\alpha\mathbf{u} + \beta\mathbf{v}) \\
 &= \mathbf{u} \times \alpha\mathbf{u} + \mathbf{u} \times \beta\mathbf{v} \\
 &= \alpha(\mathbf{u} \times \mathbf{u}) + \beta(\mathbf{u} \times \mathbf{v})
 \end{aligned}$$

$$\begin{aligned}
\mathbf{v} \times \mathbf{p} &= \mathbf{v} \times (\alpha \mathbf{u} + \beta \mathbf{v}) \\
&= \mathbf{v} \times \alpha \mathbf{u} + \mathbf{v} \times \beta \mathbf{v} \\
&= \alpha(\mathbf{v} \times \mathbf{u}) + \beta(\mathbf{v} \times \mathbf{v})
\end{aligned}$$

Since any vector crossed with itself yields zero, these equations simplify to

$$\mathbf{v} \times \mathbf{p} = \alpha(\mathbf{v} \times \mathbf{u})$$

$$\mathbf{u} \times \mathbf{p} = \beta(\mathbf{u} \times \mathbf{v})$$

Now to solve for the coefficients α and β . If you're new to vector math, you might try to divide by $\mathbf{u} \times \mathbf{v}$ and $\mathbf{v} \times \mathbf{u}$, but you can't divide by vectors. Instead, we can take the dot product of both sides of the above equations with the plane normal $\mathbf{n} = \mathbf{u} \times \mathbf{v}$, reducing both sides to scalars, which we *can* divide by.

$$\mathbf{n} \cdot (\mathbf{v} \times \mathbf{p}) = \mathbf{n} \cdot \alpha(\mathbf{v} \times \mathbf{u})$$

$$\mathbf{n} \cdot (\mathbf{u} \times \mathbf{p}) = \mathbf{n} \cdot \beta(\mathbf{u} \times \mathbf{v})$$

Now isolating the coefficients is a simple matter of division:

$$\alpha = \frac{\mathbf{n} \cdot (\mathbf{v} \times \mathbf{p})}{\mathbf{n} \cdot (\mathbf{v} \times \mathbf{u})}$$

$$\beta = \frac{\mathbf{n} \cdot (\mathbf{u} \times \mathbf{p})}{\mathbf{n} \cdot (\mathbf{u} \times \mathbf{v})}$$

Reversing the cross products for both the numerator and denominator of α (recall that $\mathbf{a} \times \mathbf{b} = -\mathbf{b} \times \mathbf{a}$) gives us a common denominator for both coefficients:

$$\alpha = \frac{\mathbf{n} \cdot (\mathbf{p} \times \mathbf{v})}{\mathbf{n} \cdot (\mathbf{u} \times \mathbf{v})}$$

$$\beta = \frac{\mathbf{n} \cdot (\mathbf{u} \times \mathbf{p})}{\mathbf{n} \cdot (\mathbf{u} \times \mathbf{v})}$$

Now we can perform one final simplification, computing a vector $\mathbf{w}$ that will be constant for the plane's basis frame, for any planar point $\mathbf{P}$:

$$\mathbf{w} = \frac{\mathbf{n}}{\mathbf{n} \cdot (\mathbf{u} \times \mathbf{v})} = \frac{\mathbf{n}}{\mathbf{n} \cdot \mathbf{n}}$$

$$\alpha = \mathbf{w} \cdot (\mathbf{p} \times \mathbf{v})$$

$$\beta = \mathbf{w} \cdot (\mathbf{u} \times \mathbf{p})$$

6.6. Interior Testing of The Intersection Using UV Coordinates

Now that we have the intersection point's planar coordinates α and β , we can easily use these to determine if the intersection point is inside the quadrilateral — that is, if the ray actually hit the quadrilateral.

The plane is divided into coordinate regions like so:

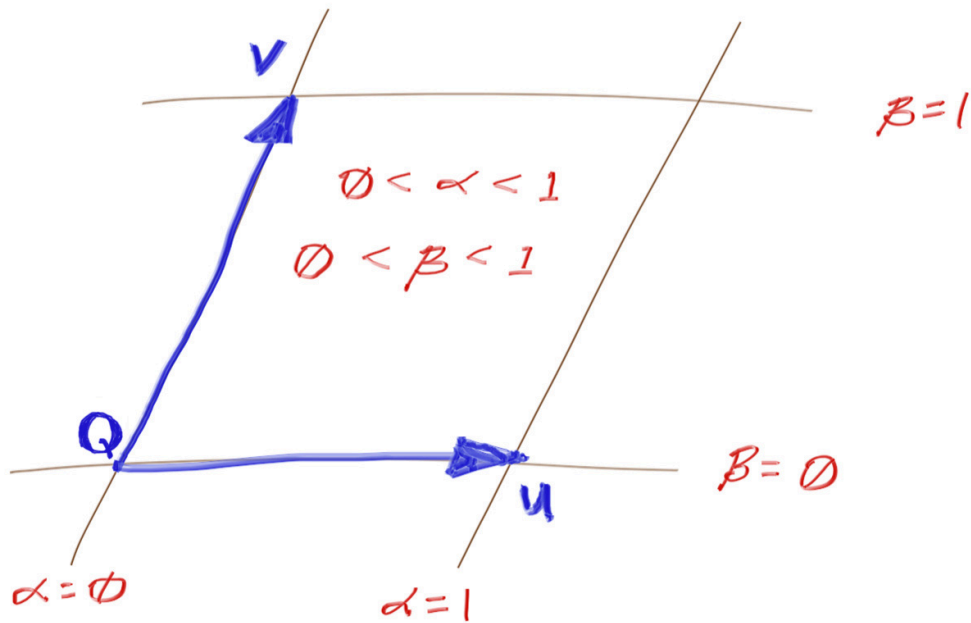


Figure 7: *Quadrilateral coordinates*

Thus, to see if a point with planar coordinates (α, β) lies inside the quadrilateral, it just needs to meet the following criteria:

1. $0 \leq \alpha \leq 1$
2. $0 \leq \beta \leq 1$

That's the last piece needed to implement quadrilateral primitives.

In order to make such experimentation a bit easier, we'll factor out the (α, β) interior test method from the hit method.

```

class quad : public hittable {
public:
    ...

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        auto denom = dot(normal, r.direction());

        // No hit if the ray is parallel to the plane.
        if (std::fabs(denom) < 1e-8)
            return false;

        // Return false if the hit point parameter t is outside the ray interval.
        auto t = (D - dot(normal, r.origin())) / denom;
        if (!ray_t.contains(t))
            return false;

        // Determine if the hit point lies within the planar shape using its plane
        coordinates.
        auto intersection = r.at(t);
        vec3 planar_hitpt_vector = intersection - Q;
        auto alpha = dot(w, cross(planar_hitpt_vector, v));
        auto beta = dot(w, cross(u, planar_hitpt_vector));

        if (!is_interior(alpha, beta, rec))
            return false;

        // Ray hits the 2D shape; set the rest of the hit record and return true.
        rec.t = t;
        rec.p = intersection;
        rec.mat = mat;
        rec.set_face_normal(r, normal);

        return true;
    }

    virtual bool is_interior(double a, double b, hit_record& rec) const {
        interval unit_interval = interval(0, 1);
        // Given the hit point in plane coordinates, return false if it is outside
        the
        // primitive, otherwise set the hit record UV coordinates and return true.

        if (!unit_interval.contains(a) || !unit_interval.contains(b))
            return false;

        rec.u = a;
        rec.v = b;
        return true;
    }

private:
    point3 Q;
    vec3 u, v;
    vec3 w;
    shared_ptr<material> mat;
    aabb bbox;
    vec3 normal;

```

```
double D;  
};
```

Listing 53: [quad.h] *Final quad class*

And now we add a new scene to demonstrate our new `quad` primitive:

```

#include "rtweekend.h"

#include "bvh.h"
#include "camera.h"
#include "hittable.h"
#include "hittable_list.h"
#include "material.h"
#include "quad.h"
#include "sphere.h"
#include "texture.h"

...

void quads() {
    hittable_list world;

    // Materials
    auto left_red    = make_shared<lambertian>(color(1.0, 0.2, 0.2));
    auto back_green  = make_shared<lambertian>(color(0.2, 1.0, 0.2));
    auto right_blue  = make_shared<lambertian>(color(0.2, 0.2, 1.0));
    auto upper_orange = make_shared<lambertian>(color(1.0, 0.5, 0.0));
    auto lower_teal   = make_shared<lambertian>(color(0.2, 0.8, 0.8));

    // Quads
    world.add(make_shared<quad>(point3(-3,-2, 5), vec3(0, 0,-4), vec3(0, 4, 0),
left_red));
    world.add(make_shared<quad>(point3(-2,-2, 0), vec3(4, 0, 0), vec3(0, 4, 0),
back_green));
    world.add(make_shared<quad>(point3( 3,-2, 1), vec3(0, 0, 4), vec3(0, 4, 0),
right_blue));
    world.add(make_shared<quad>(point3(-2, 3, 1), vec3(4, 0, 0), vec3(0, 0, 4),
upper_orange));
    world.add(make_shared<quad>(point3(-2,-3, 5), vec3(4, 0, 0), vec3(0, 0,-4),
lower_teal));

    camera cam;

    cam.aspect_ratio    = 1.0;
    cam.image_width     = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth       = 50;

    cam.vfov           = 80;
    cam.lookfrom = point3(0,0,9);
    cam.lookat   = point3(0,0,0);
    cam.vup      = vec3(0,1,0);

    cam.defocus_angle = 0;

    cam.render(world);
}

int main() {
    switch (5) {
        case 1: bouncing_spheres(); break;
        case 2: checkered_spheres(); break;
    }
}

```

```
case 3: earth(); break;
case 4: perlin_spheres(); break;
case 5: quads(); break;
}
```

Listing 54: [main.cc] *A new scene with quads*

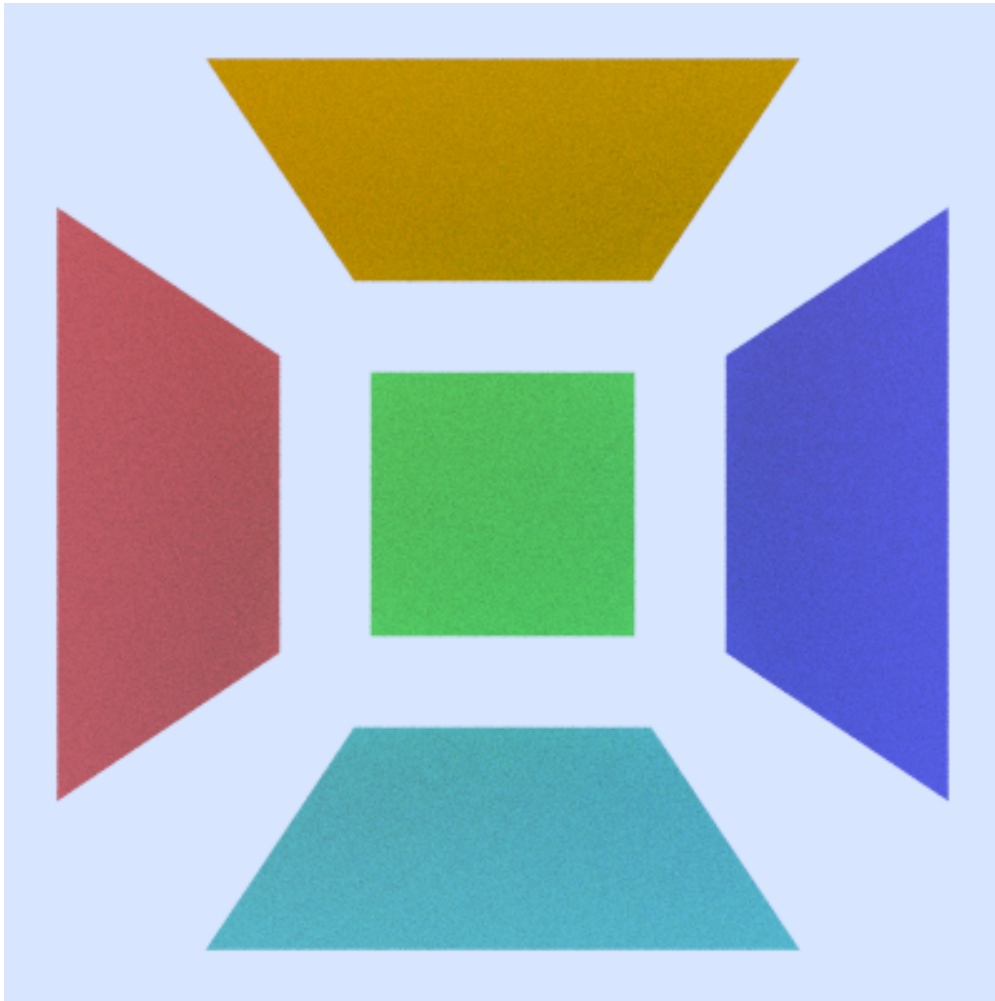


Image 16: *Quads*

6.7. Additional 2D Primitives

Pause a bit here and consider that if you use the (α, β) coordinates to determine if a point lies inside a quadrilateral (parallelogram), it's not too hard to imagine using these same 2D coordinates to determine if the intersection point lies inside *any* other 2D (planar) primitive!

For example, suppose we change the `is_interior()` function to return true if `sqrt(a*a + b*b) < r`. This would then implement disk primitives of radius `r`. For triangles, try `a > 0 && b > 0 && a + b < 1`.

We'll leave additional 2D shape possibilities as an exercise to the reader, depending on your desire to explore. You could even create cut-out stencils based on the pixels of a texture map, or a Mandelbrot shape! As a little Easter egg, check out the `alternate-2D-primitives` tag in the source repository. This has solutions for triangles, ellipses and annuli (rings) in `src/TheNextWeek/quad.h`

7. Lights

Lighting is a key component of raytracing. Early simple raytracers used abstract light sources, like points in space, or directions. Modern approaches have more physically based lights, which have position and size. To create such light sources, we need to be able to take any regular object and turn it into something that emits light into our scene.

7.1. Emissive Materials

First, let's make a light emitting material. We need to add an emitted function (we could also add it to `hit_record` instead — that's a matter of design taste). Like the background, it just tells the ray what color it is and performs no reflection. It's very simple:

```
class dielectric : public material {
    ...
}

class diffuse_light : public material {
public:
    diffuse_light(shared_ptr<texture> tex) : tex(tex) {}
    diffuse_light(const color& emit) : tex(make_shared<solid_color>(emit)) {}

    color emitted(double u, double v, const point3& p) const override {
        return tex->value(u, v, p);
    }

private:
    shared_ptr<texture> tex;
};
```

Listing 55: `[material.h]` *A diffuse light class*

So that I don't have to make all the non-emitting materials implement `emitted()`, I have the base class return black:

```
class material {
public:
    virtual ~material() = default;

    virtual color emitted(double u, double v, const point3& p) const {
        return color(0,0,0);
    }

    virtual bool scatter(
        const ray& r_in, const hit_record& rec, color& attenuation, ray& scattered
    ) const {
        return false;
    }
};
```

Listing 56: `[material.h]` *New emitted function in class material*

7.2. Adding Background Color to the Ray Color Function

Next, we want a pure black background so the only light in the scene is coming from the emitters. To do this, we'll add a background color parameter to our `ray_color` function, and pay attention to the new `color_from_emission` value.

```

class camera {
public:
    double aspect_ratio    = 1.0; // Ratio of image width over height
    int     image_width    = 100; // Rendered image width in pixel count
    int     samples_per_pixel = 10; // Count of random samples for each pixel
    int     max_depth      = 10;  // Maximum number of ray bounces into scene
    color background;      // Scene background color

    ...

private:
    ...
    color ray_color(const ray& r, int depth, const hittable& world) const {
        // If we've exceeded the ray bounce limit, no more light is gathered.
        if (depth <= 0)
            return color(0,0,0);

        hit_record rec;

        // If the ray hits nothing, return the background color.
        if (!world.hit(r, interval(0.001, infinity), rec))
            return background;

        ray scattered;
        color attenuation;
        color color_from_emission = rec.mat->emitted(rec.u, rec.v, rec.p);

        if (!rec.mat->scatter(r, rec, attenuation, scattered))
            return color_from_emission;

        color color_from_scatter = attenuation * ray_color(scattered, depth-1,
world);

        return color_from_emission + color_from_scatter;
    }
};

```

Listing 57: [camera.h] *ray_color* function with background and emitting materials

`main()` is updated to set the background color for the prior scenes:

```
void bouncing_spheres() {
    ...
    camera cam;

    cam.aspect_ratio      = 16.0 / 9.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth         = 20;
    cam.background        = color(0.70, 0.80, 1.00);
    ...
}

void checkered_spheres() {
    ...
    camera cam;

    cam.aspect_ratio      = 16.0 / 9.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth         = 50;
    cam.background        = color(0.70, 0.80, 1.00);
    ...
}

void earth() {
    ...
    camera cam;

    cam.aspect_ratio      = 16.0 / 9.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth         = 50;
    cam.background        = color(0.70, 0.80, 1.00);
    ...
}

void perlin_spheres() {
    ...
    camera cam;

    cam.aspect_ratio      = 16.0 / 9.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth         = 50;
    cam.background        = color(0.70, 0.80, 1.00);
    ...
}

void quads() {
    ...
    camera cam;

    cam.aspect_ratio      = 1.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
```

```
cam.max_depth      = 50;  
cam.background     = color(0.70, 0.80, 1.00);  
...  
}
```

Listing 58: [main.cc] *Specifying new background color*

Since we're removing the code that we used to determine the color of the sky when a ray hit it, we need to pass in a new color value for our old scene renders. We've elected to stick with a flat bluish-white for the whole sky. You could always pass in a boolean to switch between the previous skybox code versus the new solid color background. We're keeping it simple here.

7.3. Turning Objects into Lights

If we set up a rectangle as a light:

```

void simple_light() {
    hittable_list world;

    auto perertext = make_shared<noise_texture>(4);
    world.add(make_shared<sphere>(point3(0,-1000,0), 1000, make_shared<lambertian>
(perertext)));
    world.add(make_shared<sphere>(point3(0,2,0), 2, make_shared<lambertian>
(perertext)));

    auto diffflight = make_shared<diffuse_light>(color(4,4,4));
    world.add(make_shared<quad>(point3(3,1,-2), vec3(2,0,0), vec3(0,2,0),
diffflight));

    camera cam;

    cam.aspect_ratio      = 16.0 / 9.0;
    cam.image_width       = 400;
    cam.samples_per_pixel = 100;
    cam.max_depth         = 50;
    cam.background        = color(0,0,0);

    cam.vfov              = 20;
    cam.lookfrom = point3(26,3,6);
    cam.lookat   = point3(0,2,0);
    cam.vup      = vec3(0,1,0);

    cam.defocus_angle = 0;

    cam.render(world);
}

int main() {
    switch (6) {
        case 1: bouncing_spheres(); break;
        case 2: checkered_spheres(); break;
        case 3: earth(); break;
        case 4: perlin_spheres(); break;
        case 5: quads(); break;
        case 6: simple_light(); break;
    }
}

```

Listing 59: [main.cc] *A simple rectangle light*

We get:

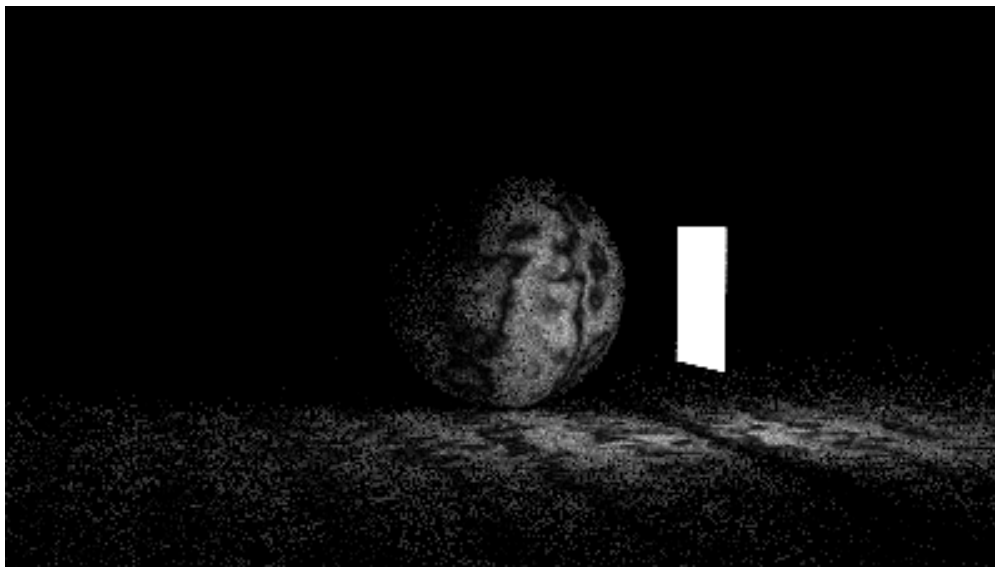


Image 17: *Scene with rectangle light source*

Note that the light is brighter than $(1, 1, 1)$. This allows it to be bright enough to light things.

Fool around with making some spheres lights too.

```
void simple_light() {  
    ...  
    auto diffflight = make_shared<diffuse_light>(color(4,4,4));  
    world.add(make_shared<sphere>(point3(0,7,0), 2, diffflight));  
    world.add(make_shared<quad>(point3(3,1,-2), vec3(2,0,0), vec3(0,2,0),  
    diffflight));  
    ...  
}
```

Listing 60: [main.cc] *A simple rectangle light plus illuminating ball*

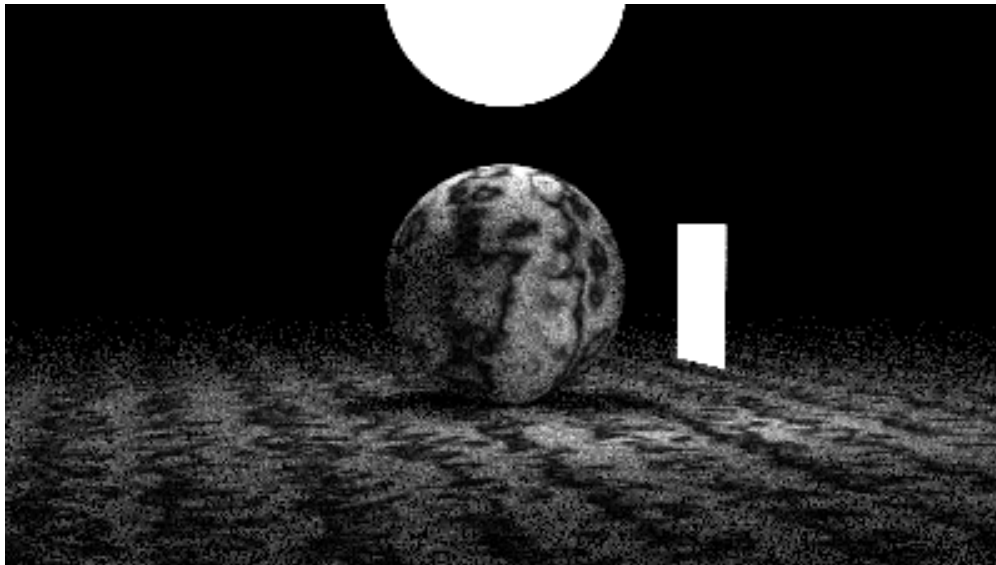


Image 18: *Scene with rectangle and sphere light sources*

7.4. Creating an Empty “Cornell Box”

The “Cornell Box” was introduced in 1984 to model the interaction of light between diffuse surfaces. Let’s make the 5 walls and the light of the box:

```

void cornell_box() {
    hittable_list world;

    auto red    = make_shared<lambertian>(color(.65, .05, .05));
    auto white  = make_shared<lambertian>(color(.73, .73, .73));
    auto green  = make_shared<lambertian>(color(.12, .45, .15));
    auto light  = make_shared<diffuse_light>(color(15, 15, 15));

    world.add(make_shared<quad>(point3(555,0,0), vec3(0,555,0), vec3(0,0,555),
green));
    world.add(make_shared<quad>(point3(0,0,0), vec3(0,555,0), vec3(0,0,555), red));
    world.add(make_shared<quad>(point3(343, 554, 332), vec3(-130,0,0),
vec3(0,0,-105), light));
    world.add(make_shared<quad>(point3(0,0,0), vec3(555,0,0), vec3(0,0,555), white));
    world.add(make_shared<quad>(point3(555,555,555), vec3(-555,0,0), vec3(0,0,-555),
white));
    world.add(make_shared<quad>(point3(0,0,555), vec3(555,0,0), vec3(0,555,0),
white));

    camera cam;

    cam.aspect_ratio    = 1.0;
    cam.image_width     = 600;
    cam.samples_per_pixel = 200;
    cam.max_depth       = 50;
    cam.background      = color(0,0,0);

    cam.vfov           = 40;
    cam.lookfrom = point3(278, 278, -800);
    cam.lookat  = point3(278, 278, 0);
    cam.vup     = vec3(0,1,0);

    cam.defocus_angle = 0;

    cam.render(world);
}

int main() {
    switch (7) {
        case 1: bouncing_spheres(); break;
        case 2: checkered_spheres(); break;
        case 3: earth(); break;
        case 4: perlin_spheres(); break;
        case 5: quads(); break;
        case 6: simple_light(); break;
        case 7: cornell_box(); break;
    }
}

```

Listing 61: [main.cc] *Cornell box scene, empty*

We get:

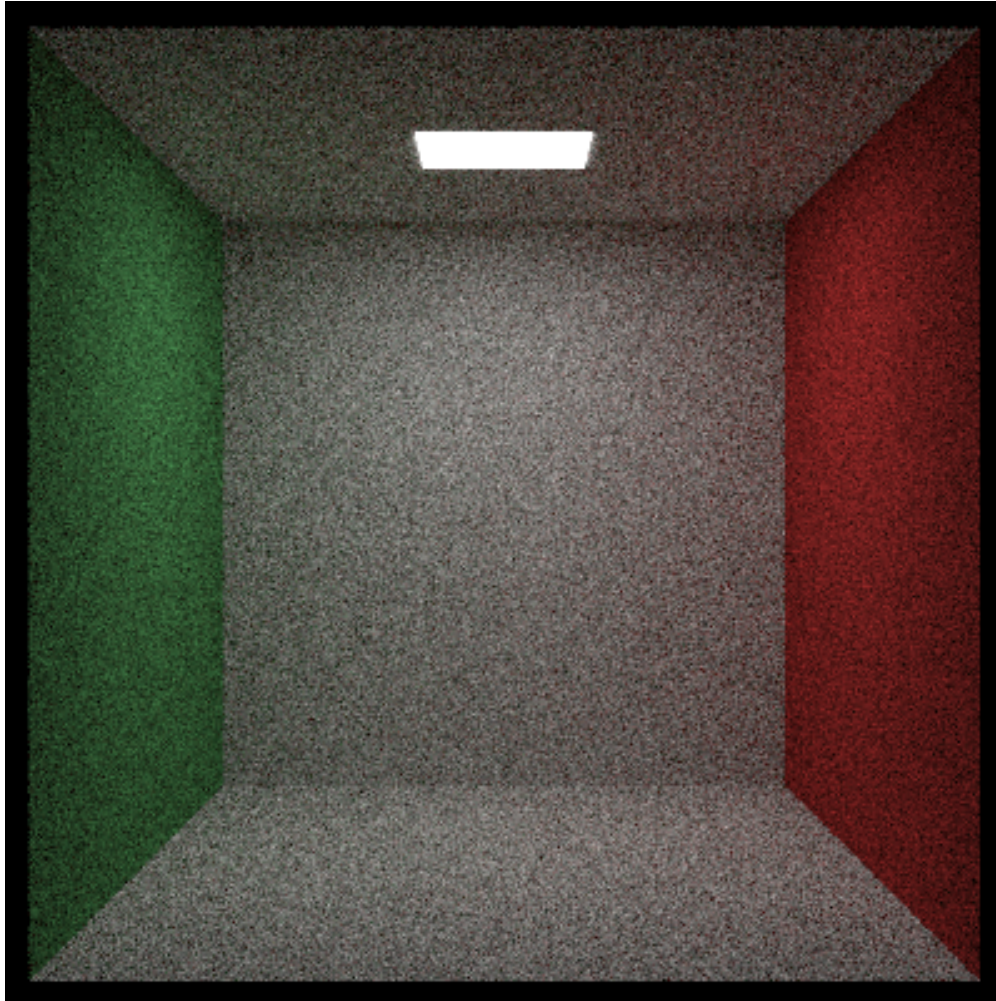


Image 19: *Empty Cornell box*

This image is very noisy because the light is small, so most random rays don't hit the light source.

8. Instances

The Cornell Box usually has two blocks in it. These are rotated relative to the walls. First, let's create a function that returns a box, by creating a `hittable_list` of six rectangles:

```

#include "hitable.h"
#include "hitable_list.h"

class quad : public hitable {
    ...
};

inline shared_ptr<hitable_list> box(const point3& a, const point3& b,
shared_ptr<material> mat)
{
    // Returns the 3D box (six sides) that contains the two opposite vertices a & b.

    auto sides = make_shared<hitable_list>();

    // Construct the two opposite vertices with the minimum and maximum coordinates.
    auto min = point3(std::fmin(a.x(),b.x()), std::fmin(a.y(),b.y()),
std::fmin(a.z(),b.z()));
    auto max = point3(std::fmax(a.x(),b.x()), std::fmax(a.y(),b.y()),
std::fmax(a.z(),b.z()));

    auto dx = vec3(max.x() - min.x(), 0, 0);
    auto dy = vec3(0, max.y() - min.y(), 0);
    auto dz = vec3(0, 0, max.z() - min.z());

    sides->add(make_shared<quad>(point3(min.x(), min.y(), max.z()), dx, dy, mat));
// front
    sides->add(make_shared<quad>(point3(max.x(), min.y(), max.z()), -dz, dy, mat));
// right
    sides->add(make_shared<quad>(point3(max.x(), min.y(), min.z()), -dx, dy, mat));
// back
    sides->add(make_shared<quad>(point3(min.x(), min.y(), min.z()), dz, dy, mat));
// left
    sides->add(make_shared<quad>(point3(min.x(), max.y(), max.z()), dx, -dz, mat));
// top
    sides->add(make_shared<quad>(point3(min.x(), min.y(), min.z()), dx, dz, mat));
// bottom

    return sides;
}

```

Listing 62: [quad. h] *A box object*

Now we can add two blocks (but not rotated).

```
void cornell_box() {  
    ...  
    world.add(make_shared<quad>(point3(555,0,0), vec3(0,555,0), vec3(0,0,555),  
green));  
    world.add(make_shared<quad>(point3(0,0,0), vec3(0,555,0), vec3(0,0,555), red));  
    world.add(make_shared<quad>(point3(343, 554, 332), vec3(-130,0,0),  
vec3(0,0,-105), light));  
    world.add(make_shared<quad>(point3(0,0,0), vec3(555,0,0), vec3(0,0,555), white));  
    world.add(make_shared<quad>(point3(555,555,555), vec3(-555,0,0), vec3(0,0,-555),  
white));  
    world.add(make_shared<quad>(point3(0,0,555), vec3(555,0,0), vec3(0,555,0),  
white));  
  
    world.add(box(point3(130, 0, 65), point3(295, 165, 230), white));  
    world.add(box(point3(265, 0, 295), point3(430, 330, 460), white));  
  
    camera cam;  
    ...  
}
```

Listing 63: [main.cc] *Adding box objects*

This gives:

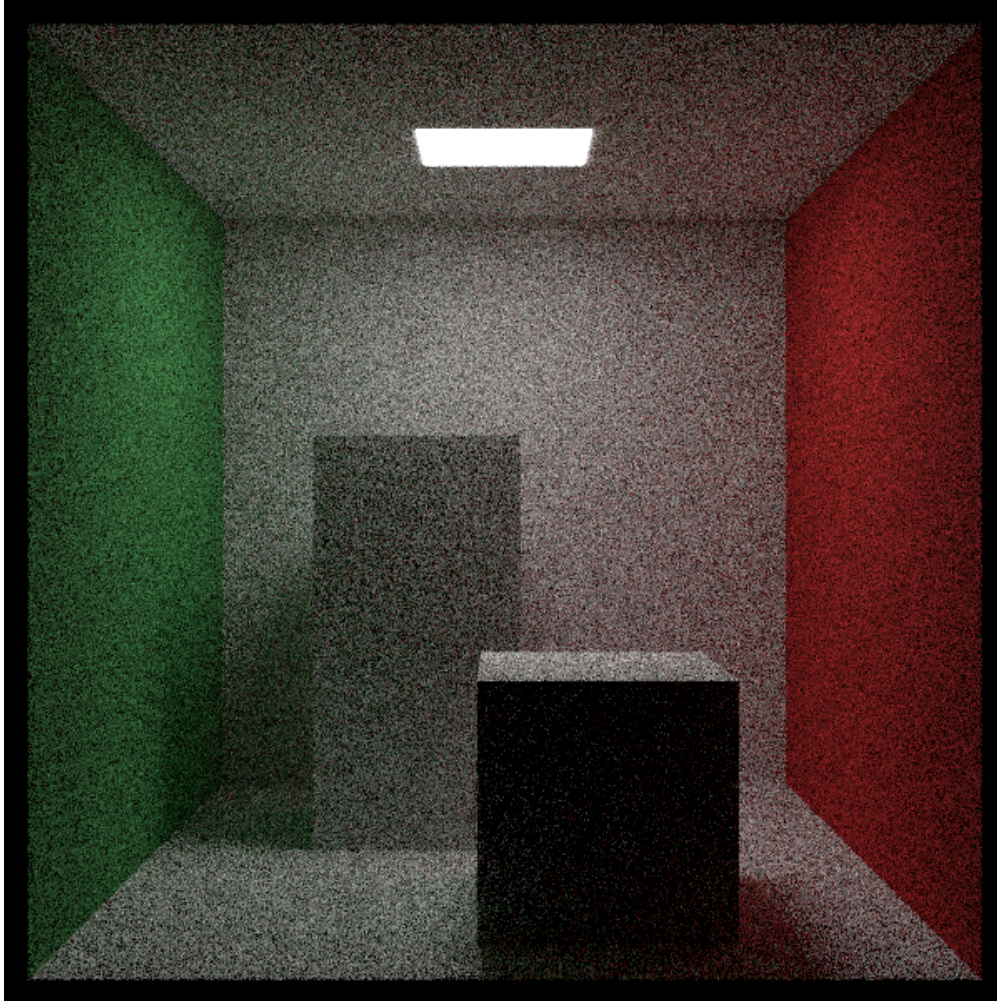


Image 20: *Cornell box with two blocks*

Now that we have boxes, we need to rotate them a bit to have them match the *real* Cornell box. In ray tracing, this is usually done with an *instance*. An instance is a copy of a geometric primitive that has been placed into the scene. This instance is entirely independent of the other copies of the primitive and can be moved or rotated. In this case, our geometric primitive is our hittable **box** object, and we want to rotate it. This is especially easy in ray tracing because we don't actually need to move objects in the scene; instead we move the rays in the opposite direction. For example, consider a *translation* (often called a *move*). We could take the pink box at the origin and add two to all its x components, or (as we almost always do in ray tracing) leave the box where it is, but in its hit routine subtract two off the x-component of the ray origin.

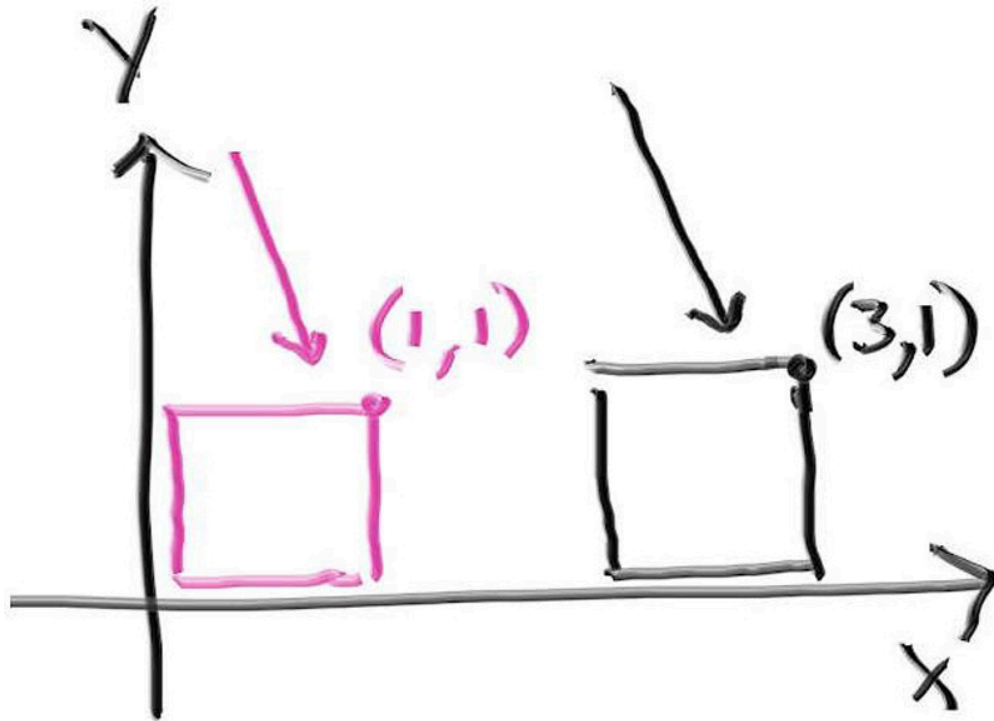


Figure 8: Ray-box intersection with moved ray vs box

8.1. Instance Translation

Whether you think of this as a move or a change of coordinates is up to you. The way to reason about this is to think of moving the incident ray backwards the offset amount, determining if an intersection occurs, and then moving that intersection point forward the offset amount.

We need to move the intersection point forward the offset amount so that the intersection is actually in the path of the incident ray. If we forgot to move the intersection point forward then the intersection would be in the path of the offset ray, which isn't correct. Let's add the code to make this happen.

```

class hittable {
    ...
};

class translate : public hittable {
public:
    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        // Move the ray backwards by the offset
        ray offset_r(r.origin() - offset, r.direction(), r.time());

        // Determine whether an intersection exists along the offset ray (and if so,
        where)
        if (!object->hit(offset_r, ray_t, rec))
            return false;

        // Move the intersection point forwards by the offset
        rec.p += offset;

        return true;
    }

private:
    shared_ptr<hittable> object;
    vec3 offset;
};

```

Listing 64: [hittable.h] *Hittable translation hit function*

... and then flesh out the rest of the `translate` class:

```

class translate : public hittable {
public:
    translate(shared_ptr<hittable> object, const vec3& offset)
        : object(object), offset(offset)
    {
        bbox = object->bounding_box() + offset;
    }

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        ...
    }

    aabb bounding_box() const override { return bbox; }

private:
    shared_ptr<hittable> object;
    vec3 offset;
    aabb bbox;
};

```

Listing 65: [hittable.h] *Hittable translation class*

We also need to remember to offset the bounding box, otherwise the incident ray might be looking in the wrong place and trivially reject the intersection. The expression `object->bounding_box() + offset` above requires some additional support.

```
class aabb {
    ...
};

const aabb aabb::empty    = aabb(interval::empty,    interval::empty,
interval::empty);
const aabb aabb::universe = aabb(interval::universe, interval::universe,
interval::universe);

aabb operator+(const aabb& bbox, const vec3& offset) {
    return aabb(bbox.x + offset.x(), bbox.y + offset.y(), bbox.z + offset.z());
}

aabb operator+(const vec3& offset, const aabb& bbox) {
    return bbox + offset;
}
```

Listing 66: [aabb.h] *The aabb + offset operator*

Since each dimension of an `aabb` is represented as an interval, we'll need to extend `interval` with an addition operator as well.

```
class interval {
    ...
};

const interval interval::empty    = interval(+infinity, -infinity);
const interval interval::universe = interval(-infinity, +infinity);

interval operator+(const interval& ival, double displacement) {
    return interval(ival.min + displacement, ival.max + displacement);
}

interval operator+(double displacement, const interval& ival) {
    return ival + displacement;
}
```

Listing 67: [interval.h] *The interval + displacement operator*

8.2. Instance Rotation

Rotation isn't quite as easy to understand or generate the formulas for. A common graphics tactic is to apply all rotations about the x, y, and z axes. These rotations are in some sense axis-aligned. First, let's rotate by θ about the z-axis. That will be changing only x and y, and in ways that don't depend on z.

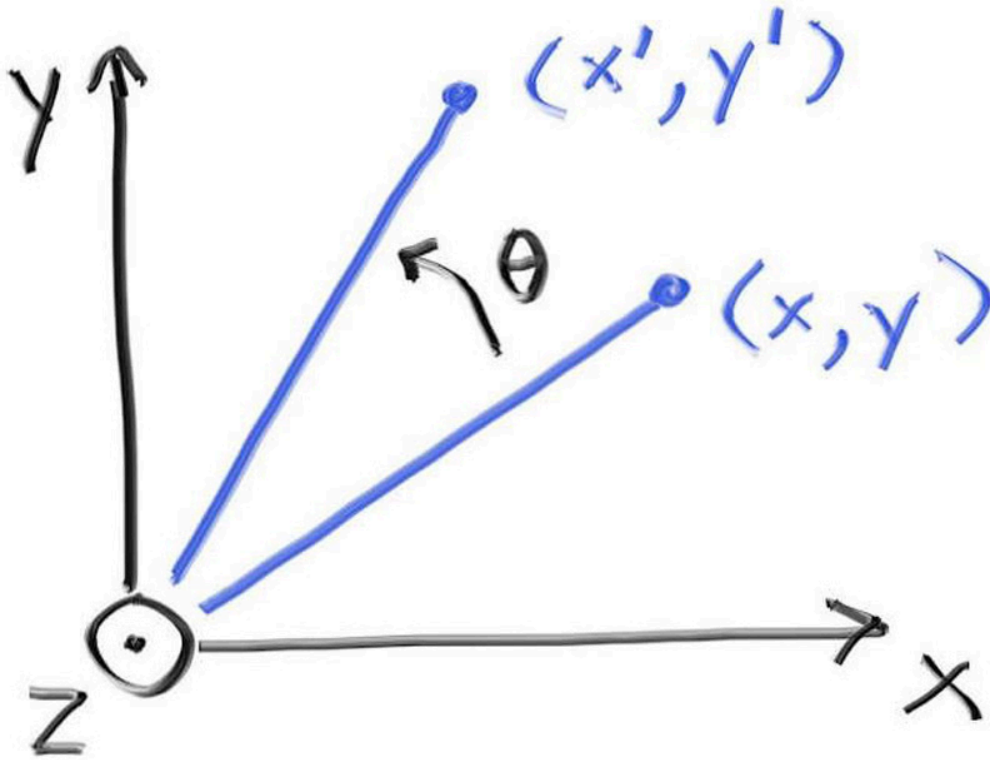


Figure 9: *Rotation about the Z axis*

This involves some basic trigonometry using formulas that I will not cover here. It's a little involved, but it is straightforward, and you can find it in any graphics text and in many lecture notes. The result for rotating counter-clockwise about z is:

$$x' = \cos(\theta) \cdot x - \sin(\theta) \cdot y$$

$$y' = \sin(\theta) \cdot x + \cos(\theta) \cdot y$$

The great thing is that it works for any θ and doesn't need any cases for quadrants or anything like that. The inverse transform is the opposite geometric operation: rotate by $-\theta$. Here, recall that $\cos(\theta) = \cos(-\theta)$ and $\sin(-\theta) = -\sin(\theta)$, so the formulas are very simple.

Similarly, for rotating about y (as we want to do for the blocks in the box) the formulas are:

$$x' = \cos(\theta) \cdot x + \sin(\theta) \cdot z$$

$$z' = -\sin(\theta) \cdot x + \cos(\theta) \cdot z$$

And if we want to rotate about the x-axis:

$$y' = \cos(\theta) \cdot y - \sin(\theta) \cdot z$$

$$z' = \sin(\theta) \cdot y + \cos(\theta) \cdot z$$

Thinking of translation as a simple movement of the initial ray is a fine way to reason about what's going on. But, for a more complex operation like a rotation, it can be easy to accidentally get your terms crossed (or forget a negative sign), so it's better to consider a rotation as a change of coordinates.

The pseudocode for the `translate::hit` function above describes the function in terms of *moving*:

1. Move the ray backwards by the offset
2. Determine whether an intersection exists along the offset ray (and if so, where)
3. Move the intersection point forwards by the offset

But this can also be thought of in terms of a *changing of coordinates*:

1. Change the ray from world space to object space
2. Determine whether an intersection exists in object space (and if so, where)
3. Change the intersection point from object space to world space

Rotating an object will not only change the point of intersection, but will also change the surface normal vector, which will change the direction of reflections and refractions. So we need to change the normal as well. Fortunately, the normal will rotate similarly to a vector, so we can use the same formulas as above. While normals and vectors may appear identical for an object undergoing rotation and translation, an object undergoing scaling requires special attention to keep the normals orthogonal to the surface. We won't cover that here, but you should research surface normal transformations if you implement scaling.

We need to start by changing the ray from world space to object space, which for rotation means rotating by $-\theta$.

$$x' = \cos(\theta) \cdot x - \sin(\theta) \cdot z$$

$$z' = \sin(\theta) \cdot x + \cos(\theta) \cdot z$$

We can now create a class for y-rotation. Let's tackle the hit function first:

```
class translate : public hittable {
    ...
};

class rotate_y : public hittable {
public:

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {

        // Transform the ray from world space to object space.

        auto origin = point3(
            (cos_theta * r.origin().x()) - (sin_theta * r.origin().z()),
            r.origin().y(),
            (sin_theta * r.origin().x()) + (cos_theta * r.origin().z())
        );

        auto direction = vec3(
            (cos_theta * r.direction().x()) - (sin_theta * r.direction().z()),
            r.direction().y(),
            (sin_theta * r.direction().x()) + (cos_theta * r.direction().z())
        );

        ray rotated_r(origin, direction, r.time());

        // Determine whether an intersection exists in object space (and if so,
        where).

        if (!object->hit(rotated_r, ray_t, rec))
            return false;

        // Transform the intersection from object space back to world space.

        rec.p = point3(
            (cos_theta * rec.p.x()) + (sin_theta * rec.p.z()),
            rec.p.y(),
            (-sin_theta * rec.p.x()) + (cos_theta * rec.p.z())
        );

        rec.normal = vec3(
            (cos_theta * rec.normal.x()) + (sin_theta * rec.normal.z()),
            rec.normal.y(),
            (-sin_theta * rec.normal.x()) + (cos_theta * rec.normal.z())
        );

        return true;
    }
};
```

Listing 68: [hittable.h] *Hittable rotate-Y hit function*

... and now for the rest of the class:

```
class rotate_y : public hittable {
public:
    rotate_y(shared_ptr<hittable> object, double angle) : object(object) {
        auto radians = degrees_to_radians(angle);
        sin_theta = std::sin(radians);
        cos_theta = std::cos(radians);
        bbox = object->bounding_box();

        point3 min( infinity,  infinity,  infinity);
        point3 max(-infinity, -infinity, -infinity);

        for (int i = 0; i < 2; i++) {
            for (int j = 0; j < 2; j++) {
                for (int k = 0; k < 2; k++) {
                    auto x = i*bbox.x.max + (1-i)*bbox.x.min;
                    auto y = j*bbox.y.max + (1-j)*bbox.y.min;
                    auto z = k*bbox.z.max + (1-k)*bbox.z.min;

                    auto newx = cos_theta*x + sin_theta*z;
                    auto newz = -sin_theta*x + cos_theta*z;

                    vec3 tester(newx, y, newz);

                    for (int c = 0; c < 3; c++) {
                        min[c] = std::fmin(min[c], tester[c]);
                        max[c] = std::fmax(max[c], tester[c]);
                    }
                }
            }
        }

        bbox = aabb(min, max);
    }

    bool hit(const ray& r, interval ray_t, hit_record& rec) const override {
        ...
    }

    aabb bounding_box() const override { return bbox; }

private:
    shared_ptr<hittable> object;
    double sin_theta;
    double cos_theta;
    aabb bbox;
};
```

Listing 69: [hittable.h] *Hittable rotate-Y class*

And the changes to Cornell are:

```
void cornell_box() {  
    ...  
    world.add(make_shared<quad>(point3(0,0,555), vec3(555,0,0), vec3(0,555,0),  
    white));  
  
    shared_ptr<hitable> box1 = box(point3(0,0,0), point3(165,330,165), white);  
    box1 = make_shared<rotate_y>(box1, 15);  
    box1 = make_shared<translate>(box1, vec3(265,0,295));  
    world.add(box1);  
  
    shared_ptr<hitable> box2 = box(point3(0,0,0), point3(165,165,165), white);  
    box2 = make_shared<rotate_y>(box2, -18);  
    box2 = make_shared<translate>(box2, vec3(130,0,65));  
    world.add(box2);  
  
    camera cam;  
    ...  
}
```

Listing 70: [main.cc] *Cornell scene with Y-rotated boxes*

Which yields:

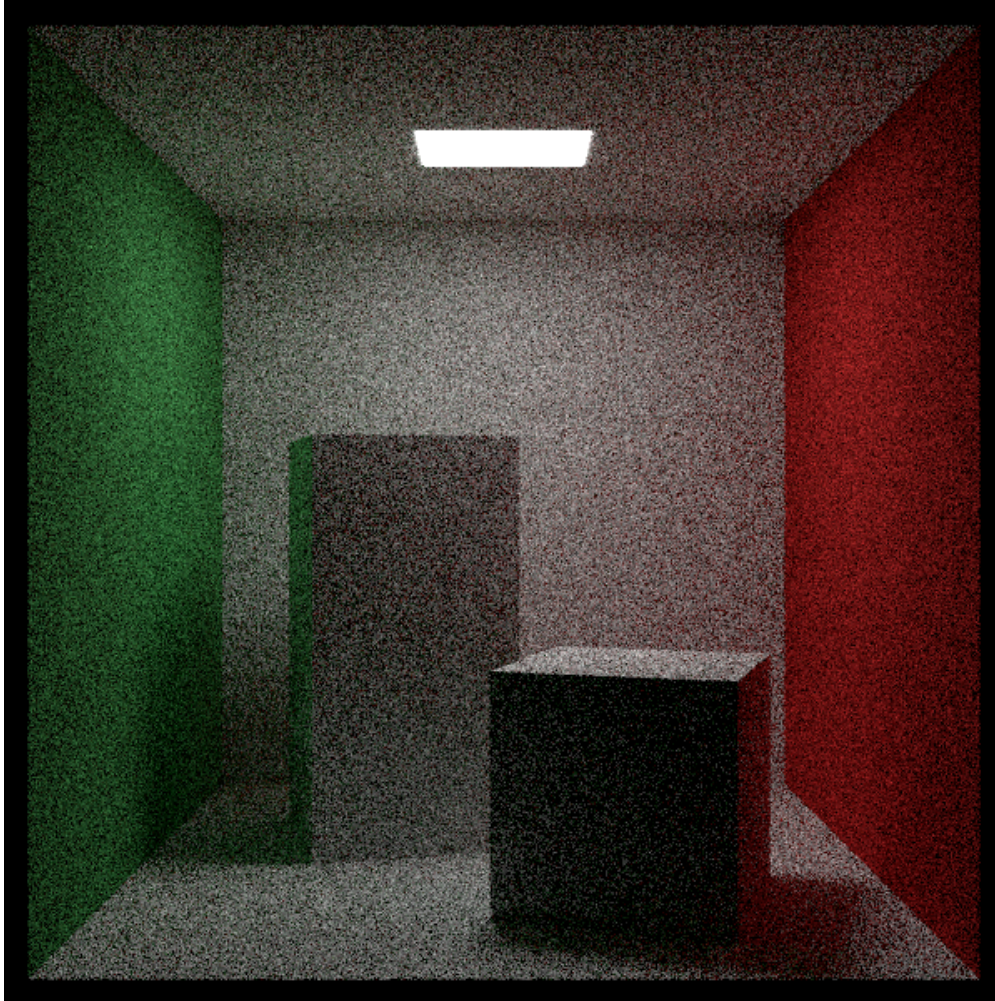


Image 21: *Standard Cornell box scene*

9. Volumes

One thing it's nice to add to a ray tracer is smoke/fog/mist. These are sometimes called *volumes* or *participating media*. Another feature that is nice to add is subsurface scattering, which is sort of like dense fog inside an object. This usually adds software architectural mayhem because volumes are a different animal than surfaces, but a cute technique is to make a volume a random surface. A bunch of smoke can be replaced with a surface that probabilistically might or might not be there at every point in the volume. This will make more sense when you see the code.

9.1. Constant Density Mediums

First, let's start with a volume of constant density. A ray going through there can either scatter inside the volume, or it can make it all the way through like the middle ray in the figure. More thin